

Multi-Container Runtime with Kernel Memory Monitor

S. Kishore (PES1UG24CS607)
Darshan Doddamani (PES1UG24CS566)

1 Introduction

This project implements a custom container runtime environment featuring a user-space supervisor and a specialized Linux kernel module for real-time memory monitoring and enforcement. The system provides process isolation via Linux namespaces and manages container lifecycles through a centralized engine.

2 Build and Setup Instructions

2.1 Prerequisites

The implementation is designed for **Ubuntu 22.04/24.04** environments. It requires kernel headers for module compilation and **Secure Boot** must be disabled to load the custom kernel module.

```
1          sudo apt update
2          sudo apt install -y build-essential linux-headers-$(
    uname -r)
3
```

2.2 Build and Load

The project uses a structured Makefile to compile the supervisor, kernel monitor, and test workloads simultaneously.

```
1          # Build all components
2          make -C boilerplate
3
4          # Load the kernel monitor
5          sudo insmod boilerplate/monitor.ko
6
```

3 System Architecture & Demo Demos

3.1 Multi-Container Supervision

The supervisor process creates isolated environments using `clone()` with `CLONE_NEWPID`, `CLONE_NEWNS`, `CLONE_NEWUTS`, and `CLONE_NEWNET` flags.

```

darshan@darshan-VirtualBox:~/Downloads/OS_JACKFRUIT-main/boilerplate$ make
gcc -O2 -Wall -Wextra -Wno-stringop-truncation -Wno-format-truncation -o engine engine.c -lpthread
gcc -O2 -Wall -static -o memory_hog memory_hog.c
gcc -O2 -Wall -static -o cpu_hog cpu_hog.c
gcc -O2 -Wall -static -o io_pulse io_pulse.c
make -C /lib/modules/6.14.0-37-generic/build M=/home/darshan/Downloads/OS_JACKFRUIT-main/boilerplate
make[1]: Entering directory '/usr/src/linux-headers-6.14.0-37-generic'
make[2]: Entering directory '/home/darshan/Downloads/OS_JACKFRUIT-main/boilerplate'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
CC [M] monitor.o
MODPOST Module.symvers
CC [M] monitor.mod.o
CC [M] .module-common.o
LD [M] monitor.ko
BTF [M] monitor.ko
Skipping BTF generation for monitor.ko due to unavailability of vmlinux
make[2]: Leaving directory '/home/darshan/Downloads/OS_JACKFRUIT-main/boilerplate'
make[1]: Leaving directory '/usr/src/linux-headers-6.14.0-37-generic'

```

Figure 1: Demo 1: Concurrent execution of 'alpha' and 'beta' containers.

3.2 Metadata and State Tracking

The supervisor tracks the state of every container, including its PID, memory limits, and scheduling priority (nice value).

```

darshan@darshan-VirtualBox:~/Downloads/OS_JACKFRUIT-main/boilerplate$ sudo ./engine ps

```

ID	PID	STATE	STARTED	SOFT_MIB	HARD_MIB	EXIT	SIGNAL	NICE	ROOTFS	COMMAND	LOG
beta	7435	exited	2026-04-20 11:09:50	40	64	127	0	0	/home/darshan/Downloads/OS_JACKFRUIT-main/boilerplate/rootfs-b		
eta	/cpu_hog	30								logs/beta.log	
alpha	7430	exited	2026-04-20 11:09:50	40	64	127	0	0	/home/darshan/Downloads/OS_JACKFRUIT-main/boilerplate/rootfs-a		
loha	/cpu_hog	30								logs/alpha.log	

Figure 2: Demo 2: Metadata tracking via 'ps' command output.

3.3 Logging Pipeline

Logs are managed via a bounded-buffer producer/consumer model. Producers read from container pipes and a single logger thread flushes data to per-container files.

```

[INFO] === DEMO 1: ps (empty) ===
ID PID STATE STARTED SOFT_MIB HARD_MIB EXIT SIGNAL NICE ROOTFS COMMAND LOG
(no containers tracked)
[INFO] === DEMO 1: start alpha + beta (multi-container) ===
Container 'alpha' started with pid 8505
Container 'beta' started with pid 8508

```

Figure 3: Demo 3: Bounded-buffer logging contents for separate containers.

```

[INFO] === DEMO 3: logs alpha ===
cpu_hog alive elapsed=1 accumulator=8392087432302535450
cpu_hog alive elapsed=2 accumulator=11497287402919248430
cpu_hog alive elapsed=3 accumulator=17129244973523364602
cpu_hog alive elapsed=4 accumulator=11253685046388173260
cpu_hog alive elapsed=5 accumulator=749543332795969032
[INFO] === DEMO 3: logs beta ===
cpu_hog alive elapsed=1 accumulator=3374096630254551923
cpu_hog alive elapsed=2 accumulator=16299798625130275653
cpu_hog alive elapsed=3 accumulator=7452513440595219447
cpu_hog alive elapsed=4 accumulator=6589820919175459783
cpu_hog alive elapsed=5 accumulator=5048840178915135330

```

Figure 4: Demo 4: CLI interaction and IPC control channel via UNIX domain sockets.

3.4 Memory Limit Enforcement

The kernel module monitors RSS (Resident Set Size). It triggers a `KERN_WARNING` for soft limits and sends `SIGKILL` for hard limit violations.

```

Container 'gamma' finished: state=exited exit=0 signal=0
[PASS] run gamma completed
[INFO] === ps after run gamma ===
ID PID STATE STARTED SOFT_MIB HARD_MIB EXIT SIGNAL NICE ROOTFS COMMAND LOG
gamma 8519 exited 2026-04-20 11:19:19 40 64 0 0 0 /home/darshan/Downloads/OS_JACKFRUIT-main/rootfs-gamma /cpu_hog 5 logs/gamma.log
beta 8508 running 2026-04-20 11:19:14 40 64 -1 0 0 /home/darshan/Downloads/OS_JACKFRUIT-main/rootfs-beta /cpu_hog 30 logs/beta.log
alpha 8505 running 2026-04-20 11:19:14 40 64 -1 0 0 /home/darshan/Downloads/OS_JACKFRUIT-main/rootfs-alpha /cpu_hog 30 logs/alpha.log

```

Figure 5: Demo 5: Kernel logs showing Soft-Limit warning emission.

```

[INFO] Starting mem1 container - watch dmesg for SOFT LIMIT warning...
Container 'mem1' finished: state=hard_limit_killed exit=-1 signal=9
[PASS] mem1 finished (expected: killed by hard limit)

[INFO] dmesg (last 20 lines, look for SOFT LIMIT and HARD LIMIT):
1361.031262 [container_monitor] Registering container=gamma pid=7758 soft=41943040 hard=67108864
1361.034324 [container_monitor] Unregister request container=gamma pid=7758
1361.034331 [container_monitor] Removed container=gamma pid=7758
1577.759507 [container_monitor] Registering container=alpha pid=8386 soft=41943040 hard=67108864
1577.811339 [container_monitor] Registering container=beta pid=8391 soft=41943040 hard=67108864
1583.069470 clocksource: Long readout interval, skipping watchdog check: cs_nsec: 1001313633 wd_nsec: 1001313656
1607.479585 [container_monitor] Unregister request container=alpha pid=8386
1607.479591 [container_monitor] Removed container=alpha pid=8386
1607.480177 [container_monitor] Unregister request container=beta pid=8391
1607.480179 [container_monitor] Removed container=beta pid=8391
1652.453326 [container_monitor] Registering container=alpha pid=8505 soft=41943040 hard=67108864
1652.468944 [container_monitor] Registering container=beta pid=8508 soft=41943040 hard=67108864
1657.199658 [container_monitor] Registering container=gamma pid=8519 soft=41943040 hard=67108864
1657.202076 workqueue: blk_mq_run_work_fn hogged CPU for >10000us 19 times, consider switching to WQ_UNBOUND
1661.495949 [container_monitor] Unregister request container=gamma pid=8519
1661.495956 [container_monitor] Removed container=gamma pid=8519
1661.516893 [container_monitor] Registering container=mem1 pid=8526 soft=8388608 hard=25165824
1663.177661 [container_monitor] SOFT LIMIT container=mem1 pid=8526 rss=8986624 limit=8388608
1669.138786 [container_monitor] HARD LIMIT container=mem1 pid=8526 rss=29958144 limit=25165824
1669.141470 [container_monitor] Unregister request container=mem1 pid=8526

```

Figure 6: Demo 6: Hard-Limit enforcement and resulting 'hard_limit_killed' state.

3.5 Scheduling and Teardown

Scheduling is managed via CFS (Completely Fair Scheduler) weights derived from nice values.

```

ID PID STATE STARTED SOFT_MIB HARD_MIB EXIT SIGNAL NICE ROOTFS COMMAND LOG
mem1 8526 hard_limit_killed 2026-04-20 11:19:24 0 24 -1 9 0 /home/darshan/Downloads/OS_JACKFRUIT-main/rootfs-mem1 /memory_hog 4 1000 logs/mem1.log
gamma 8519 exited 2026-04-20 11:19:19 40 64 0 0 0 /home/darshan/Downloads/OS_JACKFRUIT-main/rootfs-gamma /cpu_hog 5 logs/gamma.log
beta 8508 running 2026-04-20 11:19:14 40 64 -1 0 0 /home/darshan/Downloads/OS_JACKFRUIT-main/rootfs-beta /cpu_hog 30 logs/beta.log
alpha 8505 running 2026-04-20 11:19:14 40 64 -1 0 0 /home/darshan/Downloads/OS_JACKFRUIT-main/rootfs-alpha /cpu_hog 30 logs/alpha.log

```

Figure 7: Demo 7: Scheduling experiment comparing nice=0 vs nice=10 workloads.

```

Container 'cpu1' started with pid 8543
Container 'cpu2' started with pid 8546
[INFO] === logs cpu1 (nice=0, should progress faster): ===
cpu_hog alive elapsed=1 accumulator=7044903550738889053
cpu_hog alive elapsed=2 accumulator=1299463866942664933
cpu_hog alive elapsed=3 accumulator=16680300478100418508
cpu_hog alive elapsed=5 accumulator=10853717151194560769
cpu_hog alive elapsed=6 accumulator=8890238264630053270
cpu_hog alive elapsed=7 accumulator=14469112430302727073
cpu_hog alive elapsed=8 accumulator=7630629378568151330
cpu_hog alive elapsed=9 accumulator=8739495863687026642
cpu_hog alive elapsed=10 accumulator=4611752444058275255
cpu_hog alive elapsed=11 accumulator=3103737601876293401
cpu_hog alive elapsed=12 accumulator=13552479569376526851
cpu_hog alive elapsed=13 accumulator=7634525976017253785
cpu_hog alive elapsed=14 accumulator=2265423884538999522
cpu_hog alive elapsed=15 accumulator=4286161056527284547
cpu_hog done duration=15 accumulator=4286161056527284547

[INFO] === logs cpu2 (nice=10, should progress slower): ===
cpu_hog alive elapsed=1 accumulator=18254703984547916488
cpu_hog alive elapsed=2 accumulator=11967026681306285339
cpu_hog alive elapsed=3 accumulator=8158242075903452704
cpu_hog alive elapsed=4 accumulator=17298570665517581090
cpu_hog alive elapsed=5 accumulator=13776232616624326259
cpu_hog alive elapsed=6 accumulator=18244392247375532246
cpu_hog alive elapsed=7 accumulator=16770800504000276165
cpu_hog alive elapsed=8 accumulator=958575069720838768
cpu_hog alive elapsed=9 accumulator=535411617946696625
cpu_hog alive elapsed=11 accumulator=8858374897879478259
cpu_hog alive elapsed=12 accumulator=14643958994969113502
cpu_hog alive elapsed=13 accumulator=13405048065752161651
cpu_hog alive elapsed=14 accumulator=13298745681068310360
cpu_hog done duration=15 accumulator=3316372213201171755

```

Figure 8: Demo 8: Clean teardown and supervisor exit sequence.

4 Engineering Analysis

4.1 Memory Management Design

RSS was chosen over Virtual Memory size because it reflects actual physical RAM consumption. Enforcement is handled in kernel-space to eliminate the race window between monitoring and intervention.

4.2 Scheduler Behavior

Using the formula $\text{weight} \approx 1024/(1.25^{\text{nice}})$, a container with nice 0 receives roughly 9x more CPU share than nice 10 under contention. In our tests, both finished in 15 seconds real-time, but their internal throughput (loop iterations) differed based on CPU availability.

5 Conclusion

The system successfully demonstrates the fundamentals of containerization: namespace-based isolation, metadata-driven lifecycle management, and kernel-level resource enforcement.